Restoring AC Feasibility of DCOPF Solutions via Parameter-Optimized Power Flow

Anonymous Author(s)

***Abstract*—The DC optimal power flow (DCOPF) is a standard tool for power system operations due to its computational efficiency and scalability. However, DCOPF dispatches are not guaranteed to satisfy the nonlinear AC power flow (ACPF) equations or operational limits. This paper develops a parameterized, differentiable ACPF restoration layer that seeks to restore AC feasibility of DCOPF dispatches. The restoration layer incorporates distributed slack for active power balancing and PV/PQ switching for reactive power regulation, both implemented through smooth differentiable surrogates with tunable parameters including participation factors, voltage setpoints, and regulation steepness. Parameter optimization is performed offline using our proposed fully smooth DCAC^{SS} formulation, enabling gradient-based training via the implicit function theorem. The learned parameters are then deployed online in both a smooth pipeline ($\text{DCAC}^{\text{SS}}_{\text{opt}}$) and a discrete pipeline ($\text{DCAC}^{\text{DD}}_{\text{opt}}$). The approach is evaluated across IEEE and PEGASE test systems using two DCOPF variants – including a lossy DCOPF. Experimental results demonstrate that both $\text{DCAC}^{\text{SS}}_{\text{opt}}$ and $\text{DCAC}^{\text{DD}}_{\text{opt}}$ consistently removes inequality-constraint violations and significantly improve iteration counts and solving times compared to their unoptimized $\text{DCAC}_{\text{init}}$ counterparts across test cases.**

***Index Terms*—DCOPF, ACPF, Learnable Parameters, Differentiable Optimization, Feasibility Restoration.**

Nomenclature

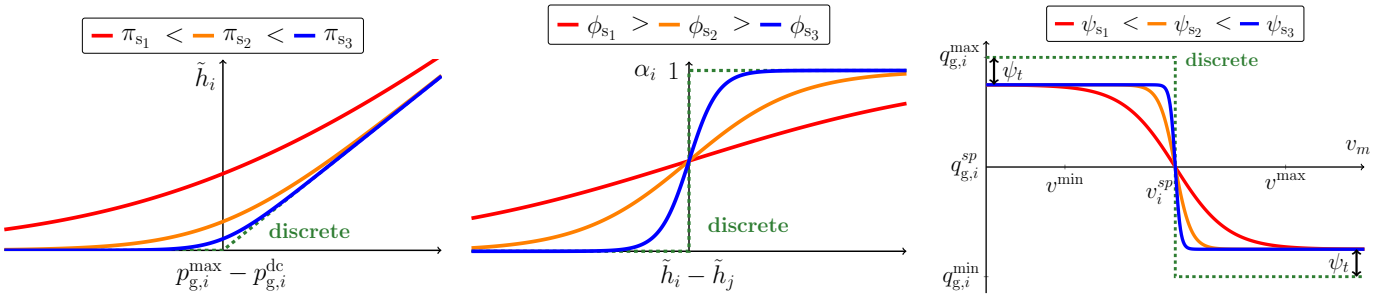
$\mathcal{N}$	Set of buses; $\mathcal{N} = \{1, \ldots, N\}$
$\mathcal{E}$	Set of lines {lines run from $i \to j$ or $j \to i$}
$\mathcal{G}$, $\mathcal{L}$	Set of generators and loads $\{\mathcal{G}, \mathcal{L} \subseteq \mathcal{N}\}$
$\mathbf{r}$, $\mathbf{b}$, $\mathbf{g}$	Line resistance, susceptance, and conductance
$\mathbf{Y}$	Complex branch line admittance
$\mathbf{\Phi}$	Line-Bus PTDF matrix, size $\mathcal{E} \times \mathcal{N}$
$\mathbf{s}_{\text{tx}}^{\max}$	Apparent power transformer limit
$\mathbf{i}_{\text{line}}^{\max}$	Current flow line limit
$\mathbf{p}_{\text{g}}^{\min}$, $\mathbf{p}_{\text{g}}^{\max}$	Active power generation limits
$\mathbf{p}_{\text{g}}$, $\mathbf{q}_{\text{g}}$	Active and reactive power generation
$\mathbf{s}_{\text{g}}$	Complex power generation; $\mathbf{p}_{\text{g}} + \mathbf{j} \cdot \mathbf{q}_{\text{g}}$
$\mathbf{p}_{\text{d}}$, $\mathbf{q}_{\text{d}}$	Active and reactive power demand
$\mathbf{q}_{\text{g}}^{\min}$, $\mathbf{q}_{\text{g}}^{\max}$	Reactive power generation limits
$\mathbf{v}^{\min}$, $\mathbf{v}^{\max}$	Voltage magnitude limits, for v_i
$\boldsymbol{\theta}^{\text{dc}}$, $\boldsymbol{\theta}^{\text{ac}}$	Voltage angles from DC and AC models
k_g, ϵ	Total losses across network, tolerance

I. Introduction

DC optimal power flow (DCOPF) serves as a core computational backbone of power-system operations, markets, and planning due to its tractability, scalability, and economic alignment with time-critical decision-making [1]. These advantages have motivated a broad class of *linearizations or DCOPF optimization formulations* that preserve the speed and structure of DC formulations while seeking improved accuracy or consistency with AC network physics. Such formulations are increasingly valuable in settings requiring repeated solves across large scenario sets, fast contingency screening, or integration within learning loops [2]–[10]. However, DCOPF solutions often violate the nonlinear AC power flow equations and/or operational constraints such as voltage limits and reactive power capabilities when these DCOPF setpoints are evaluated in an ACPF [1], [11]–[13].

To address this limitation, this paper develops a systematic approach for *AC feasibility restoration*: solving DCOPF for computational efficiency, then using AC power flow (ACPF) with distributed slack and PV/PQ switching to restore an AC feasible solution. The key innovation lies in making this restoration process differentiable and optimizable by replacing discrete control decisions with smooth parameterized surrogates. In standard ACPF restoration, convergence and runtime are strongly influenced by how active and reactive infeasibilities are handled. Single slack formulations have difficulties recovering AC feasibility from DCOPF approximations, often leading to constraint violations or infeasible solutions [11], [12], [14]. Moreover, discrete PV/PQ switching can exhibit excessive iterations due to outer loop recalculations when generators reach reactive limits, thus introducing computational inefficiency and convergence failures [15], [16]. Recent research has demonstrated the benefits of distributed slack and PV/PQ switching approaches in improving ACPF restoration of DCOPF solutions [14], [17]. Distributed slack formulations spread active power imbalances across multiple generators, helping to avoid individual generators from exceeding their limits during restoration [18], [19]. Additionally, homotopy methods and smoothing of reactive power curves can reduce iteration counts by eliminating outer loop recalculations [16], [20], [21]. Despite these advances, previous literature has typically examined distributed slack and PV/PQ switching separately, or has retained a discretized version for at least one component within AC feasibility restoration [14], [17]. As a result, the combination of smooth distributed slack with smooth PV/PQ switching in a unified differentiable framework has not been fully implemented in past work.

A further challenge is that restoration effectiveness depends critically on the selection of parameters such as participation factors and voltage setpoints. Poor parameter choices often lead to slow convergence or infeasible solutions [6], [22], [23]. To overcome these limitations, this paper proposes a supervised learning framework that optimizes restoration parameters offline using ACOPF solutions as the ground truth. The smooth DCAC^{SS} formulation enables gradient-based parameter optimization via the implicit function theorem, providing a principled way to differentiate through the restoration layer. The learned parameters are then deployed in both smooth ($\text{DCAC}^{\text{SS}}_{\text{opt}}$) and discrete ($\text{DCAC}^{\text{DD}}_{\text{opt}}$) restoration pipelines, demonstrating effective parameter transfer between formulations. This is particularly relevant in settings where



(a) Discrete vs. softplus headroom ($\tilde{h}_i$) **(b)** Discrete vs. softmax participation factor (α_i) **(c)** Discrete vs. sigmoid Q–V control curve

Fig. 1: Discrete vs. smooth AC control surrogates. (a) Softplus headroom for slack allocation, (b) softmax participation factors, and (c) differentiable sigmoid Q–V control. **Green** denotes discrete baselines and **red**/**orange**/**blue** denote increasing smooth-control steepness. The smooth controllers are tuned via (π_s, ϕ_s, ψ_s) and shaped by v^{sp} (curve shift) and ψ_t (Q-limit tolerance before $q^{\min}_{g,i}$/$q^{\max}_{g,i}$ limits engage).

TABLE I: TRAINED PARAMETERS FOR SMOOTH AC RESTORATION

Parameter (role)	Interpretation
π_s (softplus steepness)	Smooth headroom map for slack weights ($\tilde{h}_i$)
ϕ_s (softmax temperature)	Smooth participation (α_i) from headroom
ψ_s (sigmoid steepness)	Smooth Q–V regulation / PV–PQ surrogate
ψ_t (tolerance band)	Margin of reactive limits for stable switching
v^{sp} (voltage setpoint)	Tunable reference used in the Q–V surrogate

the discrete pipeline is preferred for legacy compatibility, but offline tuning is carried out in a smooth surrogate. In summary, the paper's main contributions are:

1) Developing a parameterized ACPF restoration formulation with smooth approximations for distributed slack (softplus headroom, softmax participation factors) and reactive power regulation (sigmoid Q-V curves), replacing discrete logic used in standard formulations.

2) Designing an implicit-function-theorem-based parameter learning approach for AC restoration, enabling gradient-based optimization of parameters for differentiable correction of DCOPF dispatches to AC feasible points.

3) Performing a systematic comparison of discrete versus smooth methods across multiple network cases for distributed slack allocation and reactive power regulation in ACPF, including effects of parameter optimization on both formulations.

The remainder of the paper is organized as follows. Section II presents the AC parameterization workflow. Section III reveals the DCOPF→ACPF feasibility restoration pipeline. Section IV reports the numerical results with key comparative performance metrics, and Section V concludes with future directions.

II. PARAMETERIZING THE ACPF RESTORATION LAYER

This section provides the ACPF formulation and shows how distributed slack and PV/PQ switching can provide optimizable control levers within the AC restoration process. Rather than modifying operational power system parameters, the approach optimizes internal restoration policies to enhance DCOPF→ACPF pipeline solution quality. The smooth mathematical parameterization enables gradient-based optimization of the ACPF restoration layer. Table I summarizes the learnable parameters used in this approach.

Emphasis is placed on the fact that all learned parameters are internal to the AC feasibility restoration layer trained offline to improve the DCOPF→ACPF correction; they do not modify operational Automatic Generation Control (AGC) participation factors or Automatic Voltage Regulation (AVR) setpoints in

Model 1: DC Optimal Power Flow (DCOPF) Formulation.

$$\min \sum_{i\in\mathcal{G}} \left(c_{2,i}\, p_{g,i}^2 + c_{1,i}\, p_{g,i} + c_{0,i}\right) \quad \text{(Cost Minimization)} \tag{1a}$$

$$\text{s.t. } p_{g,i} - p_{d,i} = \sum_{j\in\mathcal{E}} \overrightarrow{p_{j,i}} - \sum_{j\in\mathcal{E}} \overrightarrow{p_{i,j}} \quad \forall i \in \mathcal{N} \text{ (Balance Eqn.)} \tag{1b}$$

$$\overrightarrow{p_{i,j}} = b_{i,j}(\theta_i^{dc} - \theta_j^{dc}) \quad \forall (i,j) \in \mathcal{E} \text{ (DC Flow)} \tag{1c}$$

$$|\overrightarrow{p_{i,j}}| \le p_{i,j}^{\max} \quad \forall (i,j) \in \mathcal{E} \text{ (Line Limit)} \tag{1d}$$

$$\mathbf{p}_g^{\min} \le \mathbf{p}_g^{dc} \le \mathbf{p}_g^{\max} \quad \text{(Gen. Active Power)} \tag{1e}$$

Model 2: AC Power Flow (ACPF) with Smooth Regulation.

Power Injection Mismatches:

$$\Delta p_i = \underbrace{p_{g,i}^{dc} + \alpha_i \cdot k_g}_{\text{smooth distributed slack}} - p_{d,i} - \sum_{j\in\mathcal{N}} v_i v_j (g_{i,j}\cos\theta_{i,j} + b_{i,j}\sin\theta_{i,j}) \quad \forall i \in \mathcal{N} \tag{2a}$$

$$\Delta q_i = \underbrace{q_{g,i}^{ac}(v_i)}_{\text{smooth Q-V}} - q_{d,i} - \sum_{j\in\mathcal{N}} v_i v_j (g_{i,j}\sin\theta_{i,j} - b_{i,j}\cos\theta_{i,j}) \quad \forall i \in \mathcal{N} \tag{2b}$$

Smooth Distributed Slack (Participation):

$$\tilde{h}_i = \frac{1}{\pi_s}\ln(1 + e^{\pi_s(p_{g,i}^{\max} - p_{g,i}^{dc})}) - \frac{\ln 2}{\pi_s} \quad \forall i \in \mathcal{G} \tag{2c}$$

$$\alpha_i = \frac{e^{\phi_s \tilde{h}_i}}{\sum_{j\in\mathcal{G}} e^{\phi_s \tilde{h}_j}}, \quad \sum_{i\in\mathcal{G}} \alpha_i = 1 \quad \forall i \in \mathcal{G} \tag{2d}$$

Smooth Q-V Control: $(q_{g,i}^{\max\,/\,\min} \leftarrow q_{g,i}^{\max\,/\,\min} - \psi_t)$

$$q_{g,i}^{ac} = q_{g,i}^{\min} + \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{1 + e^{\psi_s(v_i - v_i^{sp}) + \ln\left(\frac{q_{g,i}^{\max} - q_{g,i}^{sp}}{q_{g,i}^{sp} - q_{g,i}^{\min}}\right)}} \quad \forall i \in \mathcal{G} \tag{2e}$$

Augmented Jacobian System:

$$\begin{bmatrix} \frac{\partial \Delta \mathbf{p}}{\partial \boldsymbol{\theta}} & \frac{\partial \Delta \mathbf{p}}{\partial \mathbf{v}} & \boldsymbol{\alpha} \\ \frac{\partial \Delta \mathbf{q}}{\partial \boldsymbol{\theta}} & \frac{\partial \Delta \mathbf{q}}{\partial \mathbf{v}} & \mathbf{0} \\ \mathbf{0}^T & \mathbf{0}^T & 0 \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{\theta} \\ \Delta\mathbf{v} \\ \Delta k_g \end{bmatrix} = - \begin{bmatrix} \Delta\mathbf{p} \\ \Delta\mathbf{q} \\ \ell^{\text{tot}} \end{bmatrix} \tag{2f}$$

Convergence: $\|\Delta\mathbf{p}\|_\infty, \|\Delta\mathbf{q}\|_\infty < \boldsymbol{\epsilon}$.

the actual system, nor do they model system dynamic response. This study seeks to optimize the most impactful parameters (ϕ_s, $\mathbf{v}^{\text{sp}}$) while keeping others fixed at empirically stable values.

A. Distributed Slack for Active Power Balancing

The distributed slack mechanism in Model 2 is the primary mechanism for active power balancing in AC feasibility restoration [14]. Rather than relying on a single slack bus, this approach distributes power imbalances across multiple generators based on their available headroom [18], [24]. While discrete distributed slack formulations can employ different allocation strategies (e.g., all equal or max-capacity based), earlier work has demonstrated the effectiveness of headroom-aware formulations to prevent active power violations [14]. Fig. 1a illustrates the transition from discrete to smooth headroom calculation $\tilde{h}_i$, while Fig. 1b shows the corresponding participation factor mapping α_i. The smooth formulation creates a continuous mathematical pathway: softplus activation computes generator headroom (2c), which feeds into softmax normalization for participation factors (2d). Parameters π_s and ϕ_s control the steepness of these smooth approximations, enabling gradient-based optimization while representing typical generator characteristics. The modified active power mismatch equations (2a) incorporate the participation factors $\boldsymbol{\alpha}$ directly into the augmented Jacobian system (2f), enabling simultaneous solution of slack distribution and power flow equations.

B. PV/PQ Switching for Reactive Power and Voltage Control

The smoothed PV/PQ switching shown in Model 2 addresses a fundamental challenge in AC power flow: generators must seamlessly transition between voltage control mode (PV) when within reactive limits and reactive power limit mode (PQ) when constrained [16], [20], [21]. Fig. 1c contrasts the discrete switching behavior with smooth sigmoid-based transitions. Traditional discrete switching produces discontinuous jumps at generator reactive limits $q_{g,i}^{\min\,/\,\max}$, creating convergence difficulties and preventing gradient-based parameter optimization. The proposed smooth Q-V regulation (2e) uses sigmoid functions to create a continuous model parameterized by steepness ψ_s, tolerance band ψ_t, and voltage setpoints v^{sp}. The tolerance parameter ψ_t introduces operational margins that prevent numerical instability near reactive limits. Notably, the $\ln(\cdot)$ offset in (2e) ensures that when $v_i = v_i^{\text{sp}}$, the reactive power output equals the desired setpoint $q_{g,i}^{\text{sp}}$, as shown in Appendix A. The reactive power mismatch equations (2b) seamlessly integrate the smooth Q-V control into the augmented Jacobian framework (2f) through voltage-dependent reactive power injection terms.

III. AC Feasibility Restoration of DCOPF

This section presents the DCOPF→ACPF pipeline variants considered in this work, encompassing different DCOPF formulations as dispatch sources and ACPF restoration mechanisms. The evaluation focuses on parameter optimization effects across these pipeline combinations to assess feasibility restoration improvements. The analysis covers using the supervised approach for the loss function and examines parameter sensitivity effects on restoration performance.

Algorithm 1: Training Parameters in Smooth DCAC^{SS}

Input: Training set $\mathcal{D} = \{(\mathbf{p}_{\text{d}}^{(k)}, \mathbf{q}_{\text{d}}^{(k)})\}_{k=1}^{K}$
AC reference setpoints $\{\mathbf{p}_{\text{g}}^{\text{ref},(k)}, \mathbf{v}^{\text{ref},(k)}\}$
Fixed DCOPF variant $\text{DC}(\cdot)$ producing $\mathbf{z}_{\text{dc}} = (\mathbf{p}_{\text{g}}^{\text{dc}}, \boldsymbol{\theta}^{\text{dc}})$
Initial parameters $\boldsymbol{\pi} = \{\pi_s\}$, $\boldsymbol{\phi} = \{\phi_s\}$, $\boldsymbol{\psi} = \{\psi_s, \psi_t, \mathbf{v}^{\text{sp}}\}$
Voltage limits $\mathbf{v}^{\min}, \mathbf{v}^{\max}$, gen. limits $\mathbf{p}_{\text{g}}^{\min}, \mathbf{p}_{\text{g}}^{\max}$
Output: Trained parameters $\boldsymbol{\pi}^\star, \boldsymbol{\phi}^\star, \boldsymbol{\psi}^\star$
Hyperparameters: Learning rate η;
weights w_P, w_V, w_{reg}

for *epoch* $= 1, \ldots, N_{epoch}$ **do**
  Shuffle $\mathcal{D}$ and form mini-batches $\mathcal{B}$
  foreach *batch* $\mathcal{B} \subset \mathcal{D}$ **do**
    Initialize batch loss $\mathcal{L} \leftarrow 0$
    foreach $(\mathbf{p}_{\text{d}}, \mathbf{q}_{\text{d}}) \in \mathcal{B}$ **do**
      `// 1) Forward DCOPF pass`
      $(\mathbf{p}_{\text{g}}^{\text{dc}}, \boldsymbol{\theta}^{\text{dc}}) \leftarrow \text{DCOPF}$
      `// 2) Forward ACPF pass`
      $(\mathbf{p}_{\text{g}}^{\text{ac}}, \mathbf{q}_{\text{g}}^{\text{ac}}, \mathbf{v}, \mathbf{i}) \leftarrow \text{ACPF}_{\boldsymbol{\pi},\boldsymbol{\phi},\boldsymbol{\psi}}(\mathbf{p}_{\text{d}}, \mathbf{q}_{\text{d}}, \mathbf{p}_{\text{g}}^{\text{dc}}, \boldsymbol{\theta}^{\text{dc}})$
      `// 3) Scenario loss`
      $\mathcal{L}^{(k)} \leftarrow w_P \left\| \mathbf{p}_{\text{g}}^{\text{ac}} - \mathbf{p}_{\text{g}}^{\text{ref}} \right\|_2^2 + w_V \left\| \mathbf{v} - \mathbf{v}^{\text{ref}} \right\|_2^2$
      $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}^{(k)}$
    `// 4) Regularization and updates`
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|}\mathcal{L} + w_{\text{reg}}\left(\|\boldsymbol{\psi}\|_2^2 + \|\boldsymbol{\pi}\|_2^2 + \|\boldsymbol{\phi}\|_2^2\right)$
    Compute gradients $\nabla_{\boldsymbol{\psi}}\mathcal{L}$, $\nabla_{\boldsymbol{\pi}}\mathcal{L}$, $\nabla_{\boldsymbol{\phi}}\mathcal{L}$ via backprop;
    $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \eta\nabla_{\boldsymbol{\psi}}\mathcal{L}$
    $\boldsymbol{\pi} \leftarrow \boldsymbol{\pi} - \eta\nabla_{\boldsymbol{\pi}}\mathcal{L}$
    $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \eta\nabla_{\boldsymbol{\phi}}\mathcal{L}$
return $\boldsymbol{\psi}^\star = \boldsymbol{\psi}$, $\boldsymbol{\pi}^\star = \boldsymbol{\pi}$, $\boldsymbol{\phi}^\star = \boldsymbol{\phi}$ `// Optimized parameters can be employed in either` $\text{DCAC}_{\text{opt}}^{\text{SS}}$ `or` $\text{DCAC}_{\text{opt}}^{\text{DD}}$ `pipeline`

A. Variants of the DCOPF → ACPF Pipeline

1) DCOPF Variants: Traditional lossless DCOPF neglects network losses, while loss-augmented variants attempt to incorporate them while preserving computational tractability.

The baseline DC_{BASE} represents the standard lossless DCOPF formulation detailed in Model 1. This vanilla approach assumes no transmission losses, providing optimal computational efficiency at the cost of physical realism. To improve accuracy while preserving tractability, this work also considers the Loss Quadratic Convex Program (LQCP) variant [25].

The DC_{LQCP} formulation incorporates line losses directly through convex quadratic constraints while maintaining the linear angle-flow relationship from (1c). For each transmission line (i, j) with resistance r_{ij}, losses are modeled as:

$$\overrightarrow{p_{i,j}} + \overleftarrow{p_{i,j}} \geq r_{i,j} \left(\overrightarrow{p_{i,j}}\right)^2 \tag{3}$$

where $\overrightarrow{p_{i,j}}$ and $\overleftarrow{p_{i,j}}$ represent directional power flows. This convex approximation captures loss effects while preserving the computational advantages of convex optimization, though nonlinear constraints may increase solution time [25].

2) ACPF Restoration Variants: The ACPF restoration layer employs different combinations of distributed slack and PV/PQ switching mechanisms, as illustrated in Fig. 2. Three primary variants are evaluated:

$\text{DCAC}^{\text{BASE}}$: Uses a traditional single slack bus formulation without parameter optimization, serving as a baseline for restoration performance comparison.

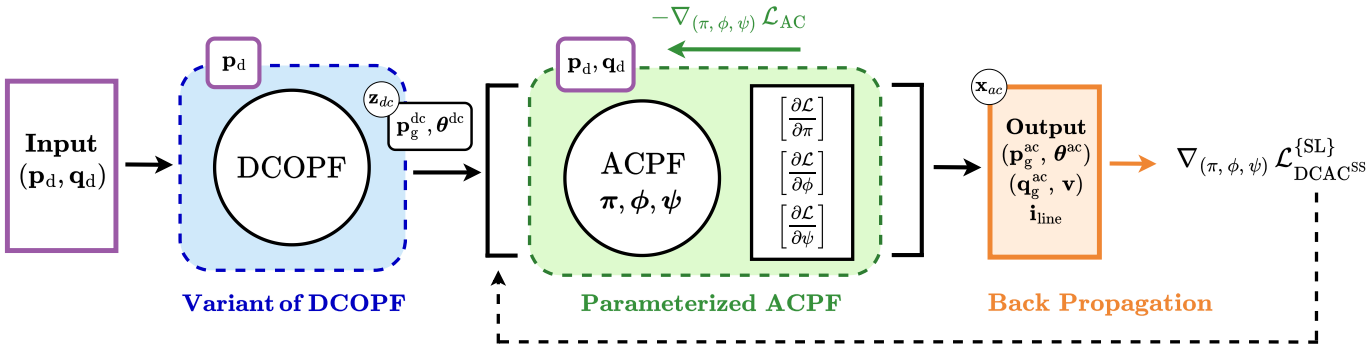


Fig. 2: **Load** inputs $(\mathbf{p}_\text{d}, \mathbf{q}_\text{d})$ feed a **DCOPF** variant (DC^{BASE} / DC^{LQCP}) via $\mathbf{p}_\text{d}$, producing DC setpoints $\mathbf{z}_{dc} = (\mathbf{p}_\text{g}^\text{dc}, \boldsymbol{\theta}^\text{dc})$. These initialize a parameterized **ACPF** that maps $(\mathbf{p}_\text{d}, \mathbf{q}_\text{d}, \mathbf{z}_{dc}) \mapsto \mathbf{x}_{ac} = \{(\mathbf{p}_\text{g}^\text{ac}, \boldsymbol{\theta}^\text{ac}), (\mathbf{q}_\text{g}^\text{ac}, \mathbf{v}), \mathbf{i}_\text{line}\}$. The ACPF contains switchable *regulation modules*: a *distributed slack* block with parameters $\boldsymbol{\pi} = \{\pi_s\}$ (softplus headroom steepness) and $\boldsymbol{\phi} = \{\phi_s\}$ (softmax participation temperature), and a *Q–V regulation* block with $\boldsymbol{\psi} = \{\psi_s, \psi_t, \mathbf{v}^\text{sp}\}$ (sigmoid steepness, tolerance, and voltage setpoint). The modules can be either *discrete* (D) or *smooth* (S) implementations, yielding DCAC^{DD}, or DCAC^{SS}. **Outputs** are trained with supervised learning (SL), using ACOPF targets.

$\text{DCAC}^{\text{SS}}_{\text{opt/init}}$: Employs the fully smooth formulation with both distributed slack and PV/PQ switching implemented via differentiable surrogates. The 'opt' variant uses parameters trained via Algorithm 1, and 'init' uses initial parameter values.

$\text{DCAC}^{\text{DD}}_{\text{opt/init}}$: Implements discrete distributed slack and discrete PV/PQ switching mechanisms. Although training occurs in the smooth DCAC^{SS} framework for differentiability, optimized parameters can be deployed in discrete implementations. Both DCOPF variants (DC_{BASE}, DC_{LQCP}) provide initial setpoints $\mathbf{z}_{\text{dc}} = (\mathbf{p}_\text{g}^\text{dc}, \boldsymbol{\theta}^\text{dc})$ to the ACPF restoration layer, creating pipeline combinations for comprehensive evaluation.

B. Smooth ACPF-based AC Restoration of DCOPF Solutions

1) Sensitivities of the DCOPF → ACPF Pipeline: The smooth ACPF restoration layer enables gradient-based parameter optimization through the implicit function theorem (IFT). Given a DCOPF initialization $\mathbf{z}_{\text{dc}}$ and reactive demand $\mathbf{q}_\text{d}$, the ACPF layer solves the nonlinear AC power-flow equations using Newton-Raphson with smooth distributed slack and smooth PV/PQ switching. At convergence (typically 4-5 iterations), the AC state $\mathbf{x}_{\text{ac}} = (\mathbf{p}_\text{g}^\text{ac}, \boldsymbol{\theta}^\text{ac}, \mathbf{q}_\text{g}^\text{ac}, \mathbf{v})$ satisfies the augmented power-balance equations $\mathbf{g}(\mathbf{x}_{\text{ac}}; \boldsymbol{\pi}, \boldsymbol{\phi}, \boldsymbol{\psi}) = \mathbf{0}$ corresponding to (2f) with parameters $\boldsymbol{\pi} = \{\pi_s\}$ for distributed slack, $\boldsymbol{\phi} = \phi_s$ for participation factors, and $\boldsymbol{\psi} = \{\psi_s, \psi_t, \mathbf{v}^\text{sp}\}$ for Q-V regulation. The IFT provides gradients of the converged ACPF solution with respect to the learnable parameters. For each parameter vector $\boldsymbol{\xi} \in \{\boldsymbol{\pi}, \boldsymbol{\phi}, \boldsymbol{\psi}\}$, the sensitivity is computed via the adjoint approach (See Appendix B) using the converged Jacobian matrix $\mathbf{J}_{\text{ac}}$ from (2f):

$$\frac{\partial \mathbf{x}_{\text{ac}}}{\partial \boldsymbol{\xi}} = -\mathbf{J}_{\text{ac}}^{-1} \frac{\partial \mathbf{g}}{\partial \boldsymbol{\xi}}. \tag{4}$$

This gradient computation leverages the augmented Jacobian structure that already incorporates participation factors $\boldsymbol{\alpha}$ in the distributed slack formulation. Future implementations could benefit from complete gradient unrolling of the Newton iterations [9], though the IFT approach provides computational efficiency by avoiding iterative differentiation.

2) Training the ACPF Layer: Parameter optimization requires careful loss function selection to balance AC feasibility, DC consistency, and computational efficiency. The DCAC^{SS} formulation serves as the appropriate differentiable pipeline for computing optimal parameters $\boldsymbol{\pi}^\star, \boldsymbol{\phi}^\star, \boldsymbol{\psi}^\star$ through Algorithm 1. These optimized parameters can subsequently be deployed in both $\text{DCAC}^{\text{SS}}_{\text{opt}}$ and $\text{DCAC}^{\text{DD}}_{\text{opt}}$ variants, enabling performance comparison against their initial parameter counterparts.

Supervised learning (SL) was used for training. AC reference setpoints $(\mathbf{p}_\text{g}^\text{ref}, \mathbf{v}^\text{ref})$ were obtained from ACOPF solutions or trusted ACPF benchmarks, and the supervised training objective matched the restored AC state to these targets:

$$\mathcal{L}_{\text{SL}} = \frac{1}{|\mathcal{B}|} \sum_{k \in \mathcal{B}} \left[w_P \|\mathbf{p}_\text{g}^{\text{ac},(k)} - \mathbf{p}_\text{g}^{\text{ref},(k)}\|_2^2 + w_V \|\mathbf{v}^{(k)} - \mathbf{v}^{\text{ref},(k)}\|_2^2 \right] \tag{5}$$

The gradient computation in Lines 15-18 of Algorithm 1 combines the loss function derivatives with the sensitivity analysis from (4), enabling efficient parameter updates via backpropagation. The loss function (5) is augmented with a regularization term $w_{\text{reg}}(\|\boldsymbol{\psi}\|_2^2 + \|\boldsymbol{\pi}\|_2^2 + \|\boldsymbol{\phi}\|_2^2)$ to prevent overfitting and improve parameter stability.

While the proposed framework enables optimization over all restoration parameters, this work focuses on optimizing the most impactful subset—participation factor steepness (ϕ_s) and voltage setpoints ($\mathbf{v}^\text{sp}$)—with comprehensive parameter optimization left for future work.

IV. Numerical Experiments and Discussions

The DCOPF→ACPF pipeline variants described in Section III were evaluated through offline parameter optimization using supervised learning. This section presents the computational setup and results. Experiments were conducted on IEEE test cases {`ieee_30`, `ieee_118`} and PEGASE networks {`pegase_1354`, `pegase_2869`} using 4,000 total samples (1,000 per case). Computations were carried out on the XXXXX high-performance computing system at XXXXX using single 24-core compute nodes equipped with 16 GB RAM. ACOPF and DCOPF variants were solved using `PowerModels.jl` [26], while the custom ACPF implementation was developed in `pandapower.py` [27].

Active power demands ($\mathbf{p}_\text{d}$) were perturbed using Gaussian multiplicative noise $\boldsymbol{\xi} \sim \mathcal{N}(1.0, (5\%)^2)$, with reactive power

($\mathbf{q}_{\mathrm{d}}$) recomputed from randomly sampled power factors in $[0.95, 1.0]$. ACOPF scenarios that failed to converge were excluded from the training dataset. The 1,000 perturbed loading scenarios per case were split into 800 training and 200 test samples. Training employed mini-batches $\mathcal{B} \subset \mathcal{D}_{\mathrm{train}}$ of size $|\mathcal{B}| = 32$, with parameters updated by minimizing the batch-averaged supervised loss from (5). For computational efficiency, only voltage setpoints $\mathbf{v}^{\mathrm{sp}}$ (Q-V regulation) and softmax temperature ϕ_s (distributed-slack participation) were optimized. Steepness parameters $\{\pi_s, \psi_s, \psi_t\}$ were confined to empirically stable intervals based on preliminary sweeps ensuring robust Newton-Raphson convergence. For example, appropriate Q-V sigmoid steepness selection reduced ACPF iterations from 35 to 6 on the `PEGASE 1354` case.

Parameter optimization was performed using L-BFGS from `scipy.optimize.minimize` with box constraints: $\phi_s \in [600, 1000]$ (participation temperature), and $\mathbf{v}^{\mathrm{sp}} \in [0.95, 1.05]$ (voltage setpoints). Loss function weights were used to select primary importance of generation dispatch and voltage profiles. Gradients were computed using custom ACPF derivatives. Optimized parameters were deployed in both $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ and $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{opt}}$ variants, compared against initial parameter values: voltage setpoints initialized to 1.0 p.u. and participation factors were zeroed out. Newton-Raphson convergence tolerances were set to 10^{-6} p.u. for power mismatches. The mean absolute error (MAE) and cost difference (CD) quantify pipeline accuracy relative to ACOPF (where G is the number of generators):

$$\mathbf{MAE} = \frac{1}{G} \left\| \mathbf{p}_{\mathrm{g}} - \mathbf{p}_{\mathrm{g}}^{\mathrm{ACOPF}} \right\|_1 \tag{6}$$

$$\mathbf{CD} = \frac{\left| \mathrm{Cost} - \mathrm{Cost}^{\mathrm{ACOPF}} \right|}{\mathrm{Cost}^{\mathrm{ACOPF}}} \cdot 100 \tag{7}$$

A. AC Feasibility Restoration Assessment with Different DCOPF→ACPF Variants

1) Constraint Violation Table Analysis: Table II demonstrates the superior violation elimination capabilities of the optimized smooth pipeline. The $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ method achieves complete feasibility restoration, eliminating all active-power, reactive-power, voltage, and line violations across all test cases and both $\mathrm{DC}_{\mathrm{BASE}}$ and $\mathrm{DC}_{\mathrm{LQCP}}$ setpoints. In contrast, $\mathrm{DCAC}^{\mathrm{BASE}}$, which employs conventional single-slack AC power flow without distributed slack or bus-type switching, exhibits substantial violations. For `IEEE 1354`, $\mathrm{DCAC}^{\mathrm{BASE}}$ shows 66 reactive-power violations with maximum magnitude 13.87 p.u. under $\mathrm{DC}_{\mathrm{BASE}}$ setpoints, and 11 line flow violations reaching 25.68% of thermal limits. The transition to lossy DCOPF setpoints ($\mathrm{DC}_{\mathrm{LQCP}}$) consistently reduces violations across all methods. For `IEEE 1354` with $\mathrm{DCAC}^{\mathrm{BASE}}$, reactive-power violations decrease from 66 to 52 violations, with maximum magnitudes reducing from 13.87 to 12.43 p.u. Similarly, line flow violations decrease from 11 to 8 violations, with maximum percentages dropping from 25.68% to 21.94%. This improvement occurs because $\mathrm{DC}_{\mathrm{LQCP}}$ incorporates loss approximations that better condition the setpoints before AC restoration.

2) Performance Table Analysis: Table III reveals significant computational advantages for the smooth pipeline variants. The $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ method serves as the reference, demonstrating substantial improvements over other approaches. For iteration count in `IEEE 1354`, $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{init}}$ requires 11 iterations compared to 5 for $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$, representing a $\mathbf{54.5}\%$ improvement. Solving time improvements are even more pronounced: $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{init}}$ requires 32.65 s compared to 5.24 s for $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$, yielding an $\mathbf{83.9}\%$ improvement. The transition to $\mathrm{DC}_{\mathrm{LQCP}}$ setpoints provides additional modest improvements across all metrics. Appendix C summarizes the DCAC solving time.

B. Key Metrics for the Proposed Pipeline ($\mathrm{DCAC}^{\mathrm{SS}}$) and the Discrete Version ($\mathrm{DCAC}^{\mathrm{DD}}$)

1) Iteration Count Figure Analysis: Figures 3 and 4 demonstrate the iteration count distributions for `PEGASE 1354` and `PEGASE 2869` across 200 scenarios using $\mathrm{DC}_{\mathrm{BASE}}$ setpoints. The $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ method (blue bars) shows most frequency concentrated at the leftmost positions, indicating superior convergence speed due to optimized participation factors (α_i) and voltage setpoints through gradient-based training. The dashed lines represent average iteration counts for each method. Even without optimization, $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{init}}$ (green) demonstrates good performance, with its average line positioned far left compared to the discrete methods. The optimized discrete version $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{opt}}$ (orange) shows clear improvements over $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{init}}$ (red), confirming that parameter optimization benefits both smooth and discrete formulations. The smooth formulations eliminate discrete bus-type switching recalculations, providing computational advantages regardless of parameter initialization.

2) Cumulative Error Plot Analysis: Figures 5 and 6 illustrate the cumulative distribution of absolute active-power errors $|p_g - p_g^{\mathrm{ACOPF}}|$ for `IEEE 118` and `PEGASE 1354` on 200 test samples. The optimized methods (blue and orange curves with dashed lines) demonstrate better alignment with ACOPF solutions. For $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ in `PEGASE 1354`, approximately $\mathbf{88}\%$ of generator setpoints achieve errors at 10^{-2} p.u. or lower, while $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{opt}}$ achieves approximately $\mathbf{87}\%$ at the same error level. The worst-case performance also improves: $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{init}}$ shows maximum errors of 1.4 p.u., while $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{opt}}$ reduces this to approximately 1.1 p.u. The $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ method has the lowest worst-case errors at approximately 0.9 p.u., demonstrating the effectiveness of the loss function in minimizing deviations from ACOPF solutions. Similar improvements are observed for `IEEE 118`, confirming the generalizability of the optimization approach across different system scales.

V. Conclusion

This paper develops a smooth, differentiable ACPF restoration layer that maps DCOPF dispatches to AC feasible operating points. Discrete switching rules are replaced with smooth surrogates—softplus/softmax distributed slack and sigmoid Q–V regulation—so the restoration parameters can be optimized with gradients via the implicit function theorem under supervised objectives. The resulting fully smooth training formulation, $\mathrm{DCAC}^{\mathrm{SS}}$, optimizes parameters that are deployed in both smooth and discrete online pipelines ($\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$, $\mathrm{DCAC}^{\mathrm{DD}}_{\mathrm{opt}}$). Experiments on `IEEE` and `PEGASE` networks show that optimization improves AC feasibility restoration,

TABLE II: CONSTRAINT VIOLATION TABLE FOR DCOPF→ACPF VARIANTS IN BASE CASE (USING DC_{BASE} VS. DC_{LQCP} SETPOINTS)

Test case	Method	DC_{BASE} setpoints								DC_{LQCP} setpoints							
		Act. Power (p.u.)		React. Power (p.u.)		Voltage (p.u.)		Line (%)		Act. Power (p.u.)		React. Power (p.u.)		Voltage (p.u.)		Line (%)	
		#viol.	Max	#viol.	Max	#viol.	Max	#viol.	Max	#viol.	Max	#viol.	Max	#viol.	Max	#viol.	Max
IEEE 30	$DCAC^{BASE}$	1	0.001	2	0.052	1	0.002	1	3.064	1	0.001	2	0.057	1	0.002	1	3.134
	$DCAC^{DD}_{init}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{DD}_{opt}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{SS}_{init}$	0	0.000	0	0.000	1	0.001	2	3.464	0	0.000	0	0.000	1	0.001	2	2.862
	$DCAC^{SS}_{opt}$	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**
IEEE 118	$DCAC^{BASE}$	1	0.003	7	1.277	0	0.000	4	13.37	1	0.003	5	0.984	0	0.000	3	12.53
	$DCAC^{DD}_{init}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{DD}_{opt}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{SS}_{init}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{SS}_{opt}$	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**
IEEE 1354	$DCAC^{BASE}$	1	0.029	66	13.87	3	0.032	11	25.68	1	0.029	52	12.43	2	0.025	8	21.94
	$DCAC^{DD}_{init}$	1	0.002	0	0.000	0	0.000	0	0.000	1	0.002	0	0.000	0	0.000	0	0.000
	$DCAC^{DD}_{opt}$	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000	0	0.000
	$DCAC^{SS}_{init}$	0	0.000	0	0.000	1	0.026	3	12.25	0	0.000	0	0.000	1	0.019	2	11.37
	$DCAC^{SS}_{opt}$	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**	**0**	**0.000**

TABLE III: PERFORMANCE TABLE FOR DCOPF→ACPF VARIANTS IN BASE CASE (USING DC_{BASE} VS. DC_{LQCP} SETPOINTS)

Test case	Method	DC_{BASE} setpoints								DC_{LQCP} setpoints							
		Cost Diff. (%)		MAE (p.u.)		Iter. Count		Solving (s)		Cost Diff. (%)		MAE (p.u.)		Iter. Count		Solving (s)	
		val.	%Improv.	val.	%Improv.	val.	%Improv.	val.	%Improv.	val.	%Improv.	val.	%Improv.	val.	%Improv.	val.	%Improv.
IEEE 30	$DCAC^{BASE}$	1.32	**78.0**	0.000	0.0	4	0.0	0.43	−68.1	1.28	**78.9**	0.000	0.0	4	0.0	0.36	−68.7
	$DCAC^{DD}_{init}$	0.31	**6.5**	0.000	0.0	7	**42.9**	5.32	**74.6**	0.29	**6.9**	0.000	0.0	6	**33.3**	4.62	**75.1**
	$DCAC^{DD}_{opt}$	0.31	**6.5**	0.001	N/A	5	**20.0**	5.19	**74.0**	0.30	**10.0**	0.001	N/A	4	0.0	4.34	**73.5**
	$DCAC^{SS}_{init}$	0.32	**9.4**	0.000	0.0	7	**42.9**	1.83	**26.2**	0.30	**10.0**	0.000	0.0	6	**33.3**	1.48	**22.3**
	$DCAC^{SS}_{opt}$	0.29	—	0.000	—	4	—	1.35	—	0.27	—	0.000	—	4	—	1.15	—
IEEE 118	$DCAC^{BASE}$	2.15	**88.8**	0.001	0.0	5	0.0	1.26	−48.4	2.05	**89.3**	0.001	0.0	5	0.0	1.02	−51.2
	$DCAC^{DD}_{init}$	0.15	−37.5	0.000	−100.0	8	**37.5**	12.67	**80.7**	0.14	−36.4	0.000	−100.0	7	**28.6**	10.84	**80.7**
	$DCAC^{DD}_{opt}$	0.16	−33.3	0.001	0.0	6	**16.7**	11.10	**78.0**	0.15	−31.8	0.001	0.0	5	0.0	9.22	**77.3**
	$DCAC^{SS}_{init}$	0.20	−16.7	0.000	−100.0	5	0.0	3.20	**23.8**	0.18	−18.2	0.000	−100.0	5	0.0	2.68	**22.0**
	$DCAC^{SS}_{opt}$	0.24	—	0.001	—	5	—	2.44	—	0.22	—	0.001	—	5	—	2.09	—
IEEE 1354	$DCAC^{BASE}$	0.42	**23.8**	0.019	**47.4**	4	−25.0	2.31	−55.9	0.39	**23.1**	0.019	**47.4**	4	−25.0	1.96	−55.3
	$DCAC^{DD}_{init}$	0.27	−15.6	0.010	0.0	11	**54.5**	32.65	**83.9**	0.25	−16.7	0.010	0.0	10	**50.0**	27.48	**84.1**
	$DCAC^{DD}_{opt}$	0.29	−9.4	0.010	0.0	9	**44.4**	21.51	**75.6**	0.27	−10.0	0.010	0.0	8	**37.5**	17.57	**75.1**
	$DCAC^{SS}_{init}$	0.28	−12.5	0.018	**44.4**	5	0.0	7.39	**29.1**	0.26	−13.3	0.018	**44.4**	5	0.0	6.23	**29.7**
	$DCAC^{SS}_{opt}$	0.32	—	0.010	—	5	—	5.24	—	0.30	—	0.010	—	5	—	4.38	—

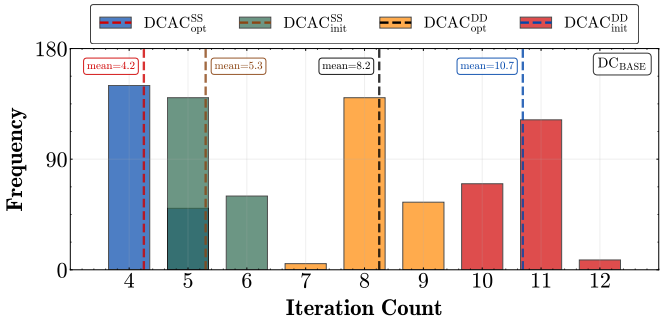


Fig. 3: Iteration count (PEGASE 1354) on 200 samples. $DCAC^{SS}_{opt}$ (**blue**), $DCAC^{SS}_{init}$ (**green**), $DCAC^{DD}_{opt}$ (**orange**), $DCAC^{DD}_{init}$ (**red**).

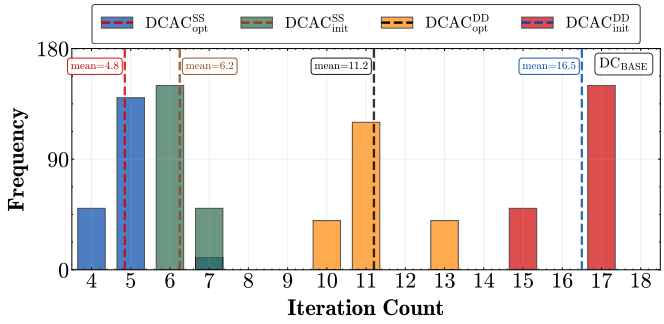


Fig. 4: Iteration count (PEGASE 2869) on 200 samples. $DCAC^{SS}_{opt}$ (**blue**), $DCAC^{SS}_{init}$ (**green**), $DCAC^{DD}_{opt}$ (**orange**), $DCAC^{DD}_{init}$ (**red**).

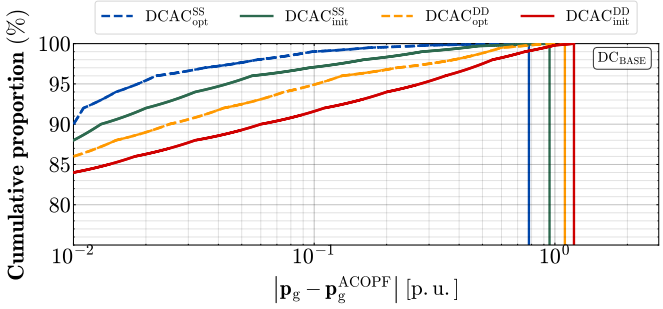


Fig. 5: Cumulative proportion of absolute active-power error (IEEE 118) on 200 test samples: $|p_g - p_g^{ACOPF}|$ for $DCAC^{SS}_{opt}$ (**blue**), $DCAC^{SS}_{init}$ (**green**), $DCAC^{DD}_{opt}$ (**orange**), $DCAC^{DD}_{init}$ (**red**).

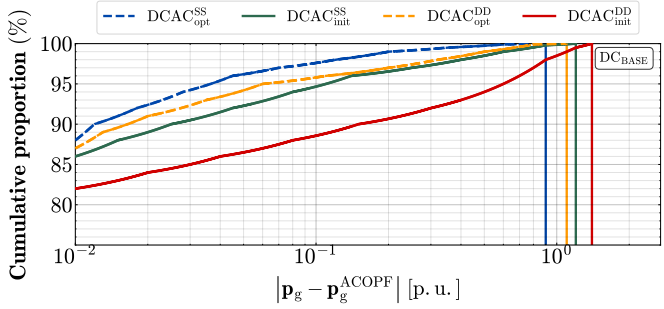


Fig. 6: Cumulative proportion of absolute active-power error (PEGASE 1354) on 200 test samples: $|p_g - p_g^{ACOPF}|$ for $DCAC^{SS}_{opt}$ (**blue**), $DCAC^{SS}_{init}$ (**green**), $DCAC^{DD}_{opt}$ (**orange**), $DCAC^{DD}_{init}$ (**red**).

reducing constraint violations and Newton iterations relative to flat start, to better restore both lossless $\mathrm{DC}_{\mathrm{BASE}}$ and lossy $\mathrm{DC}_{\mathrm{LQCP}}$ dispatches. In the 1,354-bus case, $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{opt}}$ eliminates inequality-constraint violations, decreases mean absolute error by 40%, and improves solve time by 29% relative to $\mathrm{DCAC}^{\mathrm{SS}}_{\mathrm{init}}$. Future work will incorporate economic objectives into the restoration layer, pursue end-to-end training for DCOPF→ACPF pipelines, and extend the smooth ACPF controls to transformer taps and phase shifters.

References

[1] B. Stott, J. Jardim, and O. Alsac, "DC power flow revisited," *IEEE Transactions on Power Systems*, vol. 24, no. 3, pp. 1290–1300, 2009.

[2] W. Chen, M. Tanneau, and P. Van Hentenryck, "End-to-end feasible optimization proxies for large-scale economic dispatch," *IEEE Transactions on Power Systems*, vol. 39, pp. 4723–4734, Mar. 2024.

[3] R. Ferrando, L. Pagnier, R. Mieth, Z. Liang, Y. Dvorkin, D. Bienstock, and M. Chertkov, "Physics-informed machine learning for electricity markets: A NYISO case study," *IEEE Transactions on Energy Markets, Policy and Regulation*, vol. 2, pp. 40–51, Mar. 2024.

[4] M. Li, S. Kolouri, and J. Mohammadi, "Learning to solve optimization problems with hard linear constraints," *IEEE Access*, vol. 11, pp. 59995–60004, 2023.

[5] Y. Chen and M. K. Singh, "Optimally linearizing power flow equations for improved power system dispatch," 2025.

[6] B. Taheri and D. K. Molzahn, "AC power flow feasibility restoration via a state estimation-based post-processing algorithm," *Electric Power Systems Research*, vol. 235, Oct. 2024. Presented at the *23rd Power Systems Computation Conference (PSCC)*.

[7] A. W. Rosemberg and M. Klamkin, "Differentiable optimization for deep learning-enhanced DC approximation of AC optimal power flow," *arXiv:2504.01970*, 2025.

[8] G. Constante-Flores, A. H. Quisaguano, A. J. Conejo, and C. Li, "AC-network-informed DC optimal power flow for electricity markets," *arXiv:2410.18413*, 2024.

[9] P. L. Donti, D. Rolnick, and J. Z. Kolter, "DC3: A learning method for optimization with hard constraints," in *International Conference on Learning Representations (ICLR)*, 2021.

[10] Y. Chen *et al.*, "DeepOPF-V: Voltage-constrained deep neural network for AC-OPF," *IEEE Transactions on Power Systems*, vol. 36, no. 6, pp. 4944–4956, 2021.

[11] K. Baker, "Solutions of DC OPF are never AC feasible," in *ACM e-Energy*, 2021.

[12] X. Fang, Z. Yang, J. Yu, and Y. Wang, "AC feasibility restoration in market clearing: Problem formulation and improvement," *IEEE Transactions on Industrial Informatics*, vol. 18, pp. 7597–7607, Nov. 2022.

[13] Y. Wang and Z. Yang, "Co-optimize dispatch and pricing in market clearing problem to balance conflicting market objectives," *Applied Energy*, vol. 403, p. 127065, 2026.

[14] M. A. Boateng, R. Bent, S. Misra, P. Pareek, P. Van Hentenryck, and D. Molzahn, "Towards AC feasibility of DCOPF dispatch," to appear in *Electric Power Systems Research*, 2026. To be presented at the *24th Power Systems Computation Conference (PSCC)*, June 8–12, 2026.

[15] J. Zhao, H. Chiang, P. Ju, and H. Li, "On PV–PQ bus type switching logic in power flow computation," in *16th Power Systems Computation Conference (PSCC)*, 2008.

[16] L. Zeng, H.-D. Chiang, L. S. Neves, and L. F. C. Alberto, "On the accuracy of power flow and load margin calculation caused by incorrect logical PV/PQ switching: Analytics and improved methods," *International Journal of Electrical Power & Energy Systems*, vol. 147, 2023.

[17] A. Agarwal, A. Pandey, M. Jereminov, and L. Pileggi, "Continuously differentiable analytical models for implicit control within power flow," *arXiv:1811.02000*, Nov. 2018.

[18] V. N. Bharatwaj, A. R. Abhyankar, and P. R. Bijwe, "Iterative DCOPF model using distributed slack bus," in *IEEE Power and Energy Society General Meeting*, 2012.

[19] S. V. Dhople, Y. C. Chen, A. Al-Digs, and A. D. Domínguez-García, "Reexamining the distributed slack bus," *IEEE Transactions on Power Systems*, vol. 35, no. 6, pp. 4870–4881, 2020.

[20] M. Jereminov, D. M. Bromberg, A. Pandey, M. R. Wagner, and L. Pileggi, "Evaluating feasibility within power flow," *IEEE Transactions on Smart Grid*, vol. 11, no. 4, pp. 3522–3534, 2020.

[21] T. McNamara, A. Pandey, A. Agarwal, and L. Pileggi, "Two-stage homotopy method to incorporate discrete control variables into AC-OPF," *Electric Power Systems Research*, vol. 212, p. 108283, 2022. Presented at the *22nd Power Systems Computation Conference (PSCC)*.

[22] A. Zamzam and K. Baker, "Learning optimal solutions for extremely fast ACOPF," in *IEEE SmartGridComm*, 2020.

[23] A. Van Boven and K. Baker, "Bus type switching to reduce bound violations in AC power flow," *arXiv:2511.08643*, 2025.

[24] S. V. Dhople *et al.*, "Distributed slack in AC power flow: Theory and applications," *IEEE Transactions on Power Systems*, vol. 36, no. 3, pp. 2280–2291, 2021.

[25] H. Zhao, M. Tanneau, and P. Van Hentenryck, "A linear outer approximation of line losses for DC-based optimal power flow problems," *Electric Power Systems Research*, vol. 212, p. 108272, 2022. Presented at the *22nd Power Systems Computation Conference (PSCC)*.

[26] C. Coffrin, R. Bent, K. Sundar, Y. Ng, and H. Wang, "PowerModels.jl: An open-source framework for exploring power flow formulations," in *20th Power Systems Computation Conference (PSCC)*, 2018.

[27] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, and J. M. Myrzik, "pandapower—An open-source Python tool for convenient modeling, analysis, and optimization of electric power systems," *IEEE Transactions on Power Systems*, vol. 33, p. 6510, 2018.

Appendix A
Q–V Sigmoid Curve at Voltage Setpoint

This appendix shows that the smooth Q–V regulation function is parameterized to pass exactly through the setpoint $\left(v_i^{sp},\, q_{g,i}^{sp}\right)$.

Claim: If $v_i = v_i^{sp}$, then $q_{g,i}^{ac} = q_{g,i}^{sp}$.

Q–V regulation law: Let $q_{g,i}^{\min} < q_{g,i}^{sp} < q_{g,i}^{\max}$ and $\psi_s > 0$. The smooth Q–V regulation equation is

$$q_{g,i}^{ac} = q_{g,i}^{\min} + \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{1 + \exp\left(\psi_s(v_i - v_i^{sp}) + \ln\left(\frac{q_{g,i}^{\max} - q_{g,i}^{sp}}{q_{g,i}^{sp} - q_{g,i}^{\min}}\right)\right)}. \quad (8)$$

Proof: Substituting $v_i = v_i^{sp}$ into (8) yields

$$\begin{aligned} q_{g,i}^{ac} &= q_{g,i}^{\min} + \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{1 + \exp\left(\ln\left(\frac{q_{g,i}^{\max} - q_{g,i}^{sp}}{q_{g,i}^{sp} - q_{g,i}^{\min}}\right)\right)} \\ &= q_{g,i}^{\min} + \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{1 + \dfrac{q_{g,i}^{\max} - q_{g,i}^{sp}}{q_{g,i}^{sp} - q_{g,i}^{\min}}}, \end{aligned} \quad (9)$$

where $\exp(\ln x) = x$ was used. Next,

$$1 + \frac{q_{g,i}^{\max} - q_{g,i}^{sp}}{q_{g,i}^{sp} - q_{g,i}^{\min}} = \frac{(q_{g,i}^{sp} - q_{g,i}^{\min}) + (q_{g,i}^{\max} - q_{g,i}^{sp})}{q_{g,i}^{sp} - q_{g,i}^{\min}} = \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{q_{g,i}^{sp} - q_{g,i}^{\min}}. \quad (10)$$

Substituting (10) into (9) gives

$$q_{g,i}^{ac} = q_{g,i}^{\min} + \frac{q_{g,i}^{\max} - q_{g,i}^{\min}}{\dfrac{q_{g,i}^{\max} - q_{g,i}^{\min}}{q_{g,i}^{sp} - q_{g,i}^{\min}}} = q_{g,i}^{\min} + \left(q_{g,i}^{sp} - q_{g,i}^{\min}\right) = q_{g,i}^{sp}. \quad (11)$$

Remark: Equation (8) is parameterized so that the sigmoid passes exactly through the setpoint $\left(v_i^{sp},\, q_{g,i}^{sp}\right)$ while smoothly saturating between $q_{g,i}^{\min}$ and $q_{g,i}^{\max}$.

APPENDIX B
DIFFERENTIATING THROUGH THE AUGMENTED ACPF LAYER

This appendix provides the mathematical framework for computing gradients through the augmented ACPF system to enable parameter optimization via backpropagation.

Let the converged augmented ACPF solution be $\mathbf{x}_{\text{ac}} = (\boldsymbol{\theta}, \mathbf{v}, k_g, \mathbf{p}_{\text{g}}^{\text{ac}}, \mathbf{q}_{\text{g}}^{\text{ac}})$ and define the trainable parameters

$$\boldsymbol{\xi} \triangleq \{\pi_s, \phi_s, \psi_s, \mathbf{v}^{\text{sp}}\}, \tag{12}$$

where ψ_t is treated as a fixed (or bounded) parameter that shifts effective reactive limits (see Model 2). For each scenario, $\mathbf{x}_{\text{ac}}$ satisfies the implicit system

$$\mathbf{g}(\mathbf{x}_{\text{ac}}; \boldsymbol{\xi}) = \begin{bmatrix} \Delta\mathbf{p}(\mathbf{x}_{\text{ac}}; \boldsymbol{\xi}) \\ \Delta\mathbf{q}(\mathbf{x}_{\text{ac}}; \boldsymbol{\xi}) \\ \ell^{\text{tot}}(\mathbf{x}_{\text{ac}}) \end{bmatrix} = \mathbf{0}. \tag{13}$$

Given a loss $\mathcal{L} = \frac{1}{|\mathcal{B}|}\sum_{k\in\mathcal{B}} \ell(\mathbf{x}_{\text{ac}}^{(k)}) + \mathcal{R}(\boldsymbol{\xi})$, gradients are computed via implicit differentiation. Let $\mathbf{J}_{\text{ac}} = \frac{\partial \mathbf{g}}{\partial \mathbf{x}_{\text{ac}}}$. Using the adjoint method,

$$\mathbf{J}_{\text{ac}}^{\top}\boldsymbol{\lambda} = \left(\frac{\partial \ell}{\partial \mathbf{x}_{\text{ac}}}\right)^{\top}, \qquad \frac{\partial \ell}{\partial \boldsymbol{\xi}} = -\boldsymbol{\lambda}^{\top}\frac{\partial \mathbf{g}}{\partial \boldsymbol{\xi}}. \tag{14}$$

A. Where Parameters Enter the Mismatches

Parameters affect $\Delta\mathbf{p}$ through smooth distributed slack $p_{\text{g},i}^{\text{dc}} + \alpha_i k_g$ and affect $\Delta\mathbf{q}$ through the smooth Q–V map $q_{\text{g},i}^{\text{ac}}(v_i)$:

$$\frac{\partial \Delta p_i}{\partial \alpha_i} = k_g, \qquad \frac{\partial \Delta q_i}{\partial \xi} = \frac{\partial q_{\text{g},i}^{\text{ac}}}{\partial \xi}.$$

B. Differentiable Building Blocks

a) Softplus headroom.: Let $r_i = p_{\text{g},i}^{\max} - p_{\text{g},i}^{\text{dc}}$ and $\tilde{h}_i = \frac{1}{\pi_s}\ln(1 + e^{\pi_s r_i}) - \frac{\ln 2}{\pi_s}$. Then

$$\frac{\partial \tilde{h}_i}{\partial r_i} = \sigma(\pi_s r_i), \qquad \sigma(u) = \frac{1}{1+e^{-u}}. \tag{15}$$

b) Softmax participation.: $\alpha_i = \frac{e^{\phi_s \tilde{h}_i}}{\sum_{j\in\mathcal{G}} e^{\phi_s \tilde{h}_j}}$ implies

$$\frac{\partial \alpha_i}{\partial \phi_s} = \alpha_i\Big(\tilde{h}_i - \sum_{j\in\mathcal{G}} \alpha_j \tilde{h}_j\Big), \qquad \frac{\partial \alpha_i}{\partial \tilde{h}_k} = \phi_s\, \alpha_i(\mathbb{I}[i=k] - \alpha_k). \tag{16}$$

(Chain rule gives $\partial\alpha_i/\partial\pi_s$ through $\tilde{h}$ if π_s is trained.)

c) Sigmoid Q–V regulation.: Let $q_{\text{g},i}^{\text{ac}} = q_{\min,i}^{\text{eff}} + \Delta q_i^{\text{eff}}\, s_i$ where $s_i = \left(1 + e^{\zeta_i}\right)^{-1}$ and $\zeta_i = \psi_s(v_i - v_i^{\text{sp}}) + (\cdots)$. The $(\cdots)$ term collects optional offsets (e.g., the log-offset enforcing passage through $(v_i^{\text{sp}}, q_{\text{g},i}^{\text{sp}})$) and is omitted here for brevity; including it preserves differentiability and only adds chain-rule terms (Model 2). Then $s_i(1-s_i)$ provides compact derivatives:

$$\frac{\partial q_{\text{g},i}^{\text{ac}}}{\partial \psi_s} = -\Delta q_i^{\text{eff}}\, s_i(1-s_i)(v_i - v_i^{\text{sp}}), \frac{\partial q_{\text{g},i}^{\text{ac}}}{\partial v_i^{\text{sp}}} = \Delta q_i^{\text{eff}}\, \psi_s\, s_i(1-s_i). \tag{17}$$

d) Initialization (practical note).: Parameters can be initialized to empirically stable values to promote reliable ACPF convergence; gradients are still computed from (14) at the converged solution.

APPENDIX C
TRAINING AND SOLVING TIME

This appendix presents computational performance comparisons between the ACOPF, DC variants, and DCAC.

TABLE IV: AVERAGE SOLVING TIME [S] FOR DCAC AND OPF FOR LOAD UNCERTAINTY OF $\sigma = 5\%$ (1000 SAMPLES)

	Solving Time (s)				
Test case	ACOPF	DC^{BASE}	DC^{LQCP}	$\text{DCAC}^{\text{BASE}}$	DCAC^{SS}
IEEE 30	0.052	0.067	0.104	0.30	1.21
IEEE 118	0.294	0.052	0.121	1.12	2.05
PEGASE 1354	7.703	1.270	0.292	2.19	5.24

AI Disclosure Statement: ChatGPT (version 5) was used solely for LaTeX formatting of Tables I – IV, and Grammarly's AI-tool was used for proofreading, to identify and correct grammatical errors. No other AI tools were used in this work.